WITCHERCRAFT

Minecraft Mod Documentation

A faithful recreation of The Witcher universe within Minecraft

Author: Mike

Platform: NeoForge • Language: Java

Status: In Development

1. Overview

Witchercraft is a Minecraft mod built on NeoForge that brings the world of CD Projekt Red's The Witcher 3: Wild Hunt into the Minecraft engine. The mod aims to recreate the core systems that define the Witcher experience — Signs, Alchemy, monster lore, and a deep RPG progression system — while adapting them thoughtfully to Minecraft's design language.

Rather than simply porting assets, Witchercraft treats each system as a design problem: how do you make Witcher mechanics feel meaningful and balanced inside a sandbox game that operates by fundamentally different rules? The result is a mod that respects both source materials — staying faithful to Witcher 3's feel while remaining coherent within Minecraft's world.

The mod currently implements the following major systems:

- Witcher Signs (Aard, Igni, Quen, Yrden, Axii) as active abilities
- A full Alchemy system including Potions, Decoctins, Oils, and Bombs
- Six Witcher School armor sets with custom 3D renderers
- A multi-tab Skill Tree with Combat, Signs, Alchemy, and General progression
- A Toxicity system that governs potion use and risk/reward decisions
- Bestiary and Glossary encyclopedias with monster lore entries
- A custom HUD overlay tracking all key player stats in real time
- A Meditation system for time manipulation and passive regeneration
- A custom Damage Calculator with critical hit and life steal mechanics

2. Features

2.1 Witcher Signs

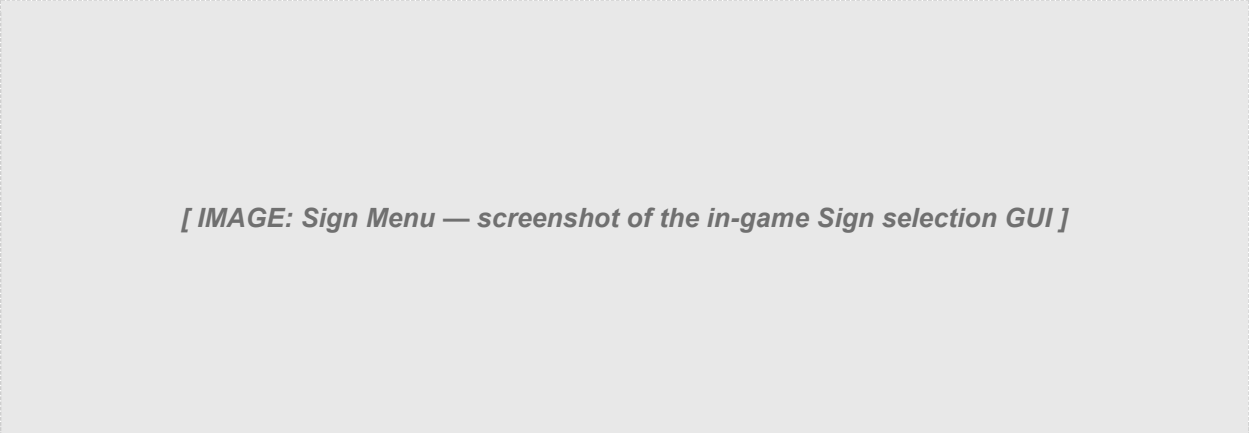

Figure 2.1 — The Sign Menu, opened with the Sign Gui keybind. Click a Sign to select it.

Signs are the signature active abilities of Witchers, powered by magic and requiring stamina to cast. Each Sign is mapped to a dedicated keybind and casting system, with a Sign GUI allowing the player to select and switch between Signs on the fly. All five canonical Signs from The Witcher 3 are implemented:

- **Aard —** Aard:

A telekinetic blast that knocks back enemies in a cone. Upgradeable to deal direct damage, increase knockback force, and expand the cone diameter.

Figure 2.2 — Aard cast in-game, showing AardParticlesEntity and knockback on target.

- **Igni —** Igni:

A stream of fire that applies burning damage on hit. Upgradeable to increase burn duration, deal additional tick damage, and boost Sign intensity.

[IMAGE: Igni in action — fire stream particles and burning enemy]

Figure 2.3 — Igni cast, showing IgniParticlesEntity fire stream.

- **Quen —** Quen:

A protective shield that absorbs incoming damage. Features a dedicated hold state with its own particle system. Upgradeable to reflect absorbed damage back to attackers and increase maximum absorption.

[IMAGE: Quen cast and hold states — showing both particle variants]

Figure 2.4 — Quen active (left) and hold state (right), each with distinct particle systems.

- **Yrden —** Yrden:

A magical trap that slows enemies who enter its radius. Upgradeable to increase the trap's effective radius and slow potency.

[IMAGE: Yrden trap placed in-game]

Figure 2.5 — Yrden trap placed, slowing a nearby mob.

- **Axii —** Axii:

A charm Sign that can manipulate enemies. Upgradeable to provide the Hero of the Village effect and increase its area of influence.

Figure 2.6 — Axii applied to a mob.

Signs use a cooldown system (Sign Cooldown effect) and a Sign Hold mechanic for charged abilities. Sign intensity, a dedicated player stat, scales all Sign effects and can be boosted by the Griffin School armor set or specific skill tree upgrades.

2.2 Alchemy System

Figure 2.7 — The Alchemy Menu. Slot 1 accepts the alchemy base; Slots 2–7 accept ingredients; Slot 8 shows the result.

The Alchemy system covers four distinct item categories that mirror The Witcher 3's approach to combat preparation. All items are craftable through a dedicated Alchemy GUI.

Potions

Potions provide temporary buffs and are the primary interaction point with the Toxicity system. The full potion roster: Swallow, Thunderbolt, Cat, Tawny Owl, Killer Whale, Blizzard, Full Moon, Golden Oriole, Petris Philter, White Raffard's Decoction, Black Blood, White Gull, White Honey.

Decoctins

Decoctins are more powerful than standard potions but carry a higher toxicity cost. Each is derived from a monster and grants abilities inspired by that creature: Ekimmara (life steal), Katakan (crit chance), Troll (out-of-combat regen), Werewolf (in-combat regen), Wyvern (attack speed),

Succubus (escalating damage), Water Hag (full-HP damage boost), Grave Hag (kill bonuses), Foglet, Leshen, Nekker Warrior, Wraith.

Oils

Oils are applied to weapons as enchantments — mapping Witcher preparation mechanics onto Minecraft's existing weapon upgrade system. Each oil targets a specific enemy archetype: Beast, Necrophage, Specter, Vampire, Insectoid, Ogroid, Draconid, Relict, Cursed, Hanged Man's Venom.

Bombs

Bombs are throwable projectile entities with unique area-of-effect behaviors: Grapeshot (fragmentation), Dancing Star (fire), Devil's Puffball (poison cloud), Dimeritium Bomb (disrupts Signs), Samum (blinds), Northern Wind (freezes).

[IMAGE: Bombs in use — one or more bombs thrown, showing area effect]

Figure 2.8 — Bomb projectile entity in flight and on impact.

2.3 Witcher School Armor Sets

All six Witcher School armor sets are implemented as full four-piece sets, each with a custom 3D renderer. Wearing a full set activates a persistent School Effect:

- Wolven Armor — School of the Wolf: balanced combat and Sign bonuses
- Feline Armor — School of the Cat: +50% critical hit damage
- Griffin Armor — School of the Griffin: +20% Sign intensity
- Ursine Armor — School of the Bear: +20% maximum health
- Viper Armor — School of the Viper: enhanced poison and stealth
- Manticore Armor — School of the Manticore: boosted Alchemy effectiveness

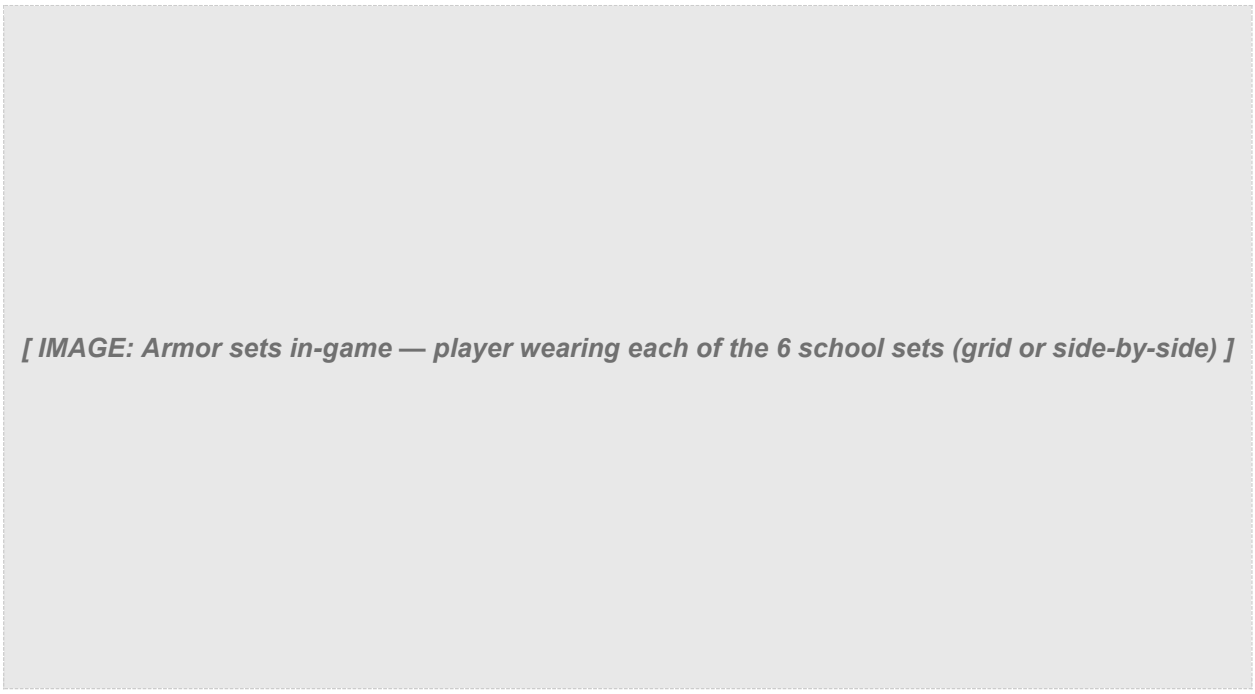

Figure 2.9 — All six Witcher School armor sets as they appear in-game. Each set uses a custom 3D renderer.

2.4 RPG Progression Systems

Player Stats & HUD

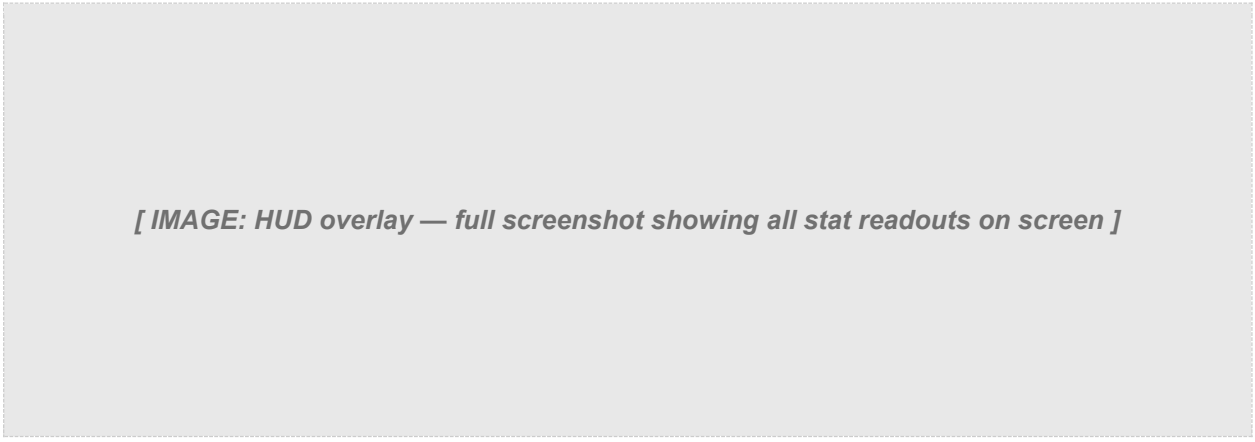

Figure 2.10 — The custom HUD overlay displaying live player stats including Toxicity, Crit Chance, Sign Intensity and more.

The following stats are tracked per-player and displayed on the HUD overlay at all times: Health, Health Regen, Stamina Regen, Attack Speed, Damage (flat and percentage), Critical Hit Chance, Critical Hit Damage, Movement Speed, Dodge Chance, Instant Kill Chance, Toxicity, Maximum Toxicity, Sign Intensity, Potion Duration, Life Steal, Reflect Damage, Oil Damage.

Skill Tree

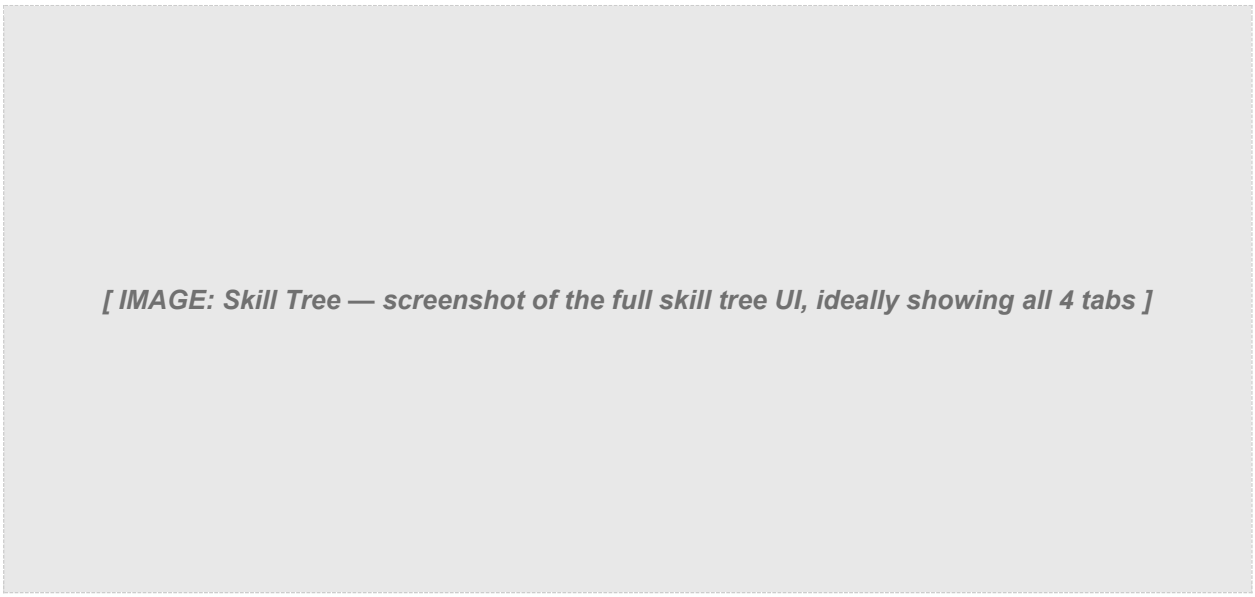

Figure 2.11 — The Skill Tree, split into Combat, Signs, Alchemy, and General tabs. Players invest points to unlock and upgrade abilities.

The Skill Tree is accessible from the custom Pause Menu and split into four tabs, each representing a different combat philosophy. Full perk details are covered in Section 4.3.

Toxicity System

Each potion and decoction consumed raises the player's current Toxicity. If Toxicity exceeds Maximum Toxicity, health begins to drain. White Honey clears all effects and resets Toxicity to zero. Both values are displayed on the HUD overlay at all times. The full mechanics are detailed in Section 4.2.

Meditation System

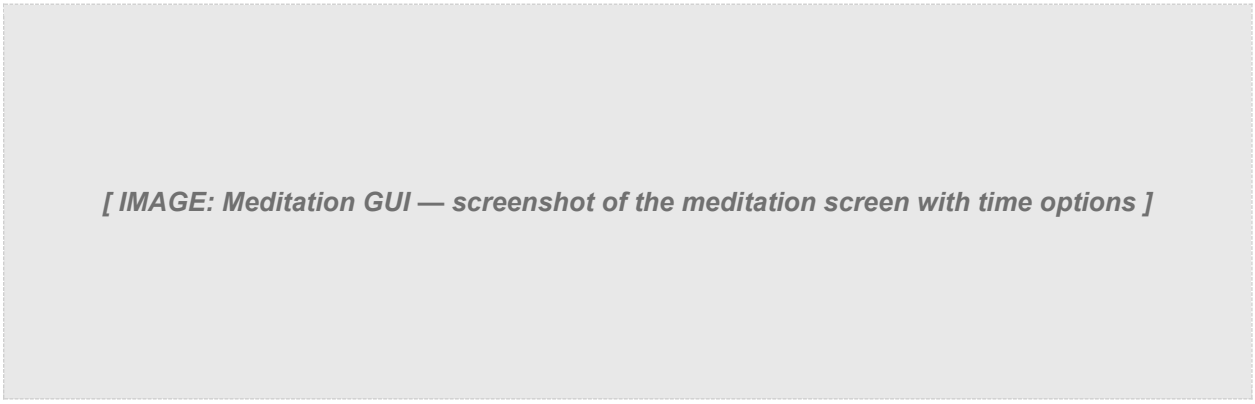

Figure 2.12 — The Meditation screen, allowing the player to fast-forward to Sunrise, Noon, Sunset, or Midnight.

The Meditation system allows fast-travel to specific times of day and enables passive health and stamina regeneration tied to the time chosen.

2.5 Bestiary & Glossary

[IMAGE: Bestiary GUI — screenshot of the in-game Bestiary showing a monster entry]

Figure 2.13 — The in-game Bestiary. Each entry shows monster type, health, attack damage, and lore.

The Bestiary is an in-game encyclopedia with entries for Drowner, Foglet, Graveir, Bruxa, Higher Vampire, and Rotfiend. The Glossary provides lore and terminology references for players unfamiliar with the Witcher universe.

3. Monsters & Mobs

A core pillar of the Witchercraft experience is its bestiary of Witcher monsters. The mod approaches monster implementation in two phases: Bestiary entries (lore and data) and entity implementation (spawnable, AI-driven mobs).

3.1 Bestiary Entries Implemented

Drowner

[IMAGE: Drowner — in-game screenshot or model render]

Amphibious necrophages that lurk near bodies of water. Drowners are among the most common monsters in the Witcher world — dangerous in groups and lethal near water. Health: 20 | Attack Damage: 5 | Type: Hostile

Foglet

[IMAGE: Foglet — in-game screenshot or model render]

Relicts that appear in areas of dense fog, using mist to disorient and ambush their prey. Health: 30 | Attack Damage: 5 | Type: Hostile

Graveir

[IMAGE: Graveir — in-game screenshot or model render]

Heavily armoured necrophages that scavenge battlefields. Their thick carapace demands careful preparation. Health: 50 | Attack Damage: 4 | Type: Hostile

Bruxa

[IMAGE: Bruxa — in-game screenshot or model render]

A higher vampire capable of taking human form, possessing exceptional speed and ferocity. Health: 30 | Attack Damage: 7 | Type: Hostile

Higher Vampire

[IMAGE: Higher Vampire — in-game screenshot or model render]

The most powerful class of vampire — immortal, intelligent, and virtually impossible to kill permanently. Health: 30 | Attack Damage: 5 | Type: Hostile

Rotfiend

[IMAGE: Rotfiend — in-game screenshot or model render]

Bloated necrophages that release a toxic gas cloud on death — a deadly trap for Witchers who close in carelessly. Health: 40 | Attack Damage: 4 | Type: Hostile

3.2 Models Ready — Entities In Development

The following monsters have completed 3D models ready for integration. Entity implementation, AI behavior, and spawning logic are currently in development:

Noonwraith

[IMAGE: Noonwraith — 3D model render or preview]

A spectral apparition born from the spirit of a woman who died violently at noon. Appears as a shimmering heat-distorted figure in open fields during midday. Planned type: Specter | Hostile

Fiend

[IMAGE: Fiend — 3D model render or preview]

A massive relict resembling a giant elk with multiple eyes, capable of hypnotizing prey with its central eye. Planned type: Relict | Hostile

Leshen

[IMAGE: Leshen — 3D model render or preview]

Ancient forest guardians that command animals and plant life, summoning crows and roots to overwhelm intruders. Planned type: Relict | Hostile

Cockatrice

[IMAGE: Cockatrice — 3D model render or preview]

A winged draconid with the head of a rooster and serpentine features. Aggressive aerial predator. Planned type: Draconid | Hostile

4. Technical Design

This section covers the key engineering and design decisions behind Witchercraft. The focus is on the design problems each system solves and the rationale behind the chosen approaches, with relevant implementation detail where it illustrates intent.

4.1 Damage Model & Formula

Witchercraft replaces Minecraft's default damage calculation with a custom DamageCalculatorProcedure that applies on every player attack. The system introduces three new variables on top of the base weapon damage: flat additional damage, percentage increased damage, critical hit chance, critical hit multiplier, and life steal.

Variables

baseDamage	The raw damage value of the weapon or attack source
additionalDamage	Flat bonus damage added before any multipliers (from skill tree or gear)
increasedDamage	Percentage damage bonus, stored as an integer (e.g. 10 = +10%)
critChance	Integer 1–100. Rolled against a random number each hit to determine if a crit occurs
critDamage	Crit damage multiplier, stored as an integer (e.g. 150 = ×1.50)
lifeSteal	Percentage of final damage restored as health, stored as an integer (e.g. 10 = 10%)

Normal Hit Formula

```
finalDamage = (baseDamage + additionalDamage) x (1 +
increasedDamage x 0.01)
healthRestored = finalDamage x lifeSteal x 0.01
```

Critical Hit Formula

A critical hit occurs when a random integer rolled between 1 and 100 is less than or equal to the player's critChance value. On a critical hit the damage is further multiplied by the critDamage multiplier:

```
critRoll = random(1, 100)
if critRoll <= critChance:
  finalDamage = (baseDamage + additionalDamage) x (1 +
increasedDamage x 0.01) x (critDamage x 0.01)
  healthRestored = finalDamage x lifeSteal x 0.01
```

Design Notes

All percentage stats are stored as integers and multiplied by 0.01 at calculation time. This avoids floating point drift in stat storage and makes values human-readable in the HUD (showing '150' for crit damage rather than '1.5'). The Ender Dragon is explicitly excluded from the critical hit path — a deliberate design decision to prevent one-shot scenarios on boss entities.

The formula is applied asynchronously via a one-tick server-side delay (queueServerWork), ensuring damage and life steal resolve correctly in multiplayer environments without race conditions on the player's health value.

4.2 Toxicity System — Full Design

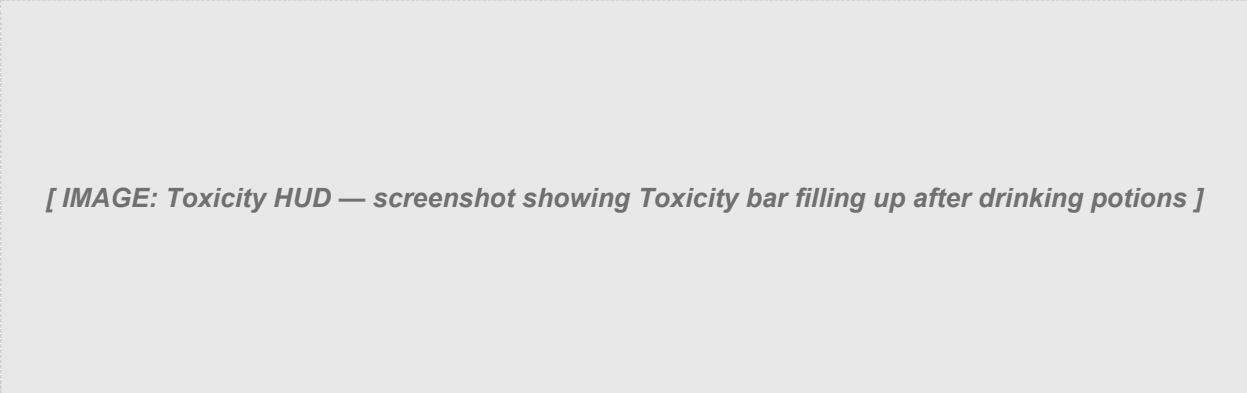

Figure 4.1 — Toxicity level rising on the HUD overlay after consecutive potion use.

The Toxicity system is one of the most mechanically significant designs in the mod. It governs the entire Alchemy pillar and creates the central risk/reward loop around potion use.

How It Works

- Every potion and decoction consumed increments the player's current Toxicity by a fixed amount specific to that item.
- Decoctins carry a higher toxicity cost than standard potions, reflecting their greater power.
- Toxicity decays passively over time — the player's Toxicity level slowly returns to zero while potions are not being consumed.
- If current Toxicity exceeds the player's Maximum Toxicity threshold, health begins to drain each tick until Toxicity falls back below the threshold.
- Drinking White Honey immediately clears all active potion and decoction effects and resets Toxicity to zero — functioning as an emergency reset at the cost of losing all active buffs.
- Current Toxicity and Maximum Toxicity are both displayed on the HUD overlay at all times, giving the player full visibility of their risk state.

Design Intent

The Toxicity system exists to prevent the dominant strategy of drinking all potions before every encounter. Without it, the optimal play is always maximum potion stacking, which trivialises the alchemy system's depth. By making over-consumption deadly, the system forces genuine preparation decisions: which potions are worth the toxicity cost for this encounter? Is a decoction worth the risk given my current toxicity level?

Maximum Toxicity can be raised through the Alchemy skill tree (Acquired Tolerance upgrades), rewarding investment in the Alchemy path with greater flexibility. This creates a meaningful progression axis — early players must be conservative, while dedicated alchemists can sustain multiple active effects simultaneously.

Acquired Tolerance Upgrades

Three tiers of Acquired Tolerance are available in the Alchemy skill tree, each increasing the player's Maximum Toxicity. This directly expands the number of simultaneous potions a player can safely maintain, rewarding commitment to the Alchemy path.

4.3 Skill Tree Architecture & Perk Design

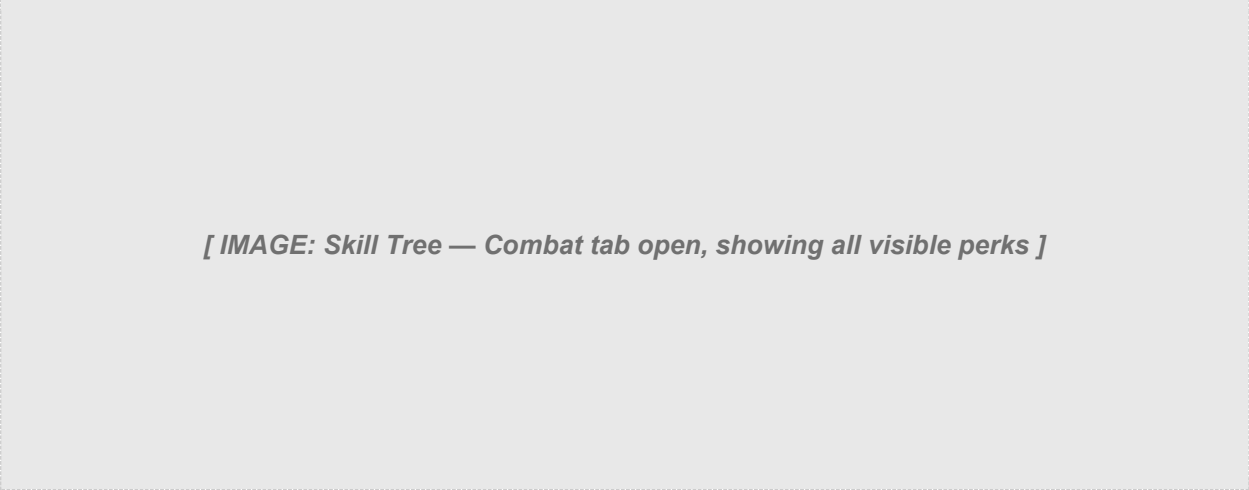

Figure 4.2 — The Combat tab of the Skill Tree. Perks are organised in tiers from basic to advanced.

The Skill Tree is organised into four independent tabs, each corresponding to a different combat philosophy: Combat, Signs, Alchemy, and General. All skill state is stored as persistent player data and synchronised between client and server via the custom network packet system, ensuring upgrades survive session restarts and function correctly in multiplayer.

Tier System

Perks within each tab are arranged in tiers. Basic perks in the lower tiers are accessible immediately, while more powerful perks in higher tiers require prior investment in the same tab. This prevents players from immediately acquiring the most impactful upgrades and encourages commitment to a chosen playstyle over the course of a playthrough.

Combat Tab Perks

The Combat tab focuses on raw offensive power and survivability:

- Muscle Memory — +3 flat damage to all attacks
- Strength Training — +10% increased damage
- Precise Blows — +12% crit chance, +75% crit damage
- Crippling Strikes — applies Bleed effect on hit

- Crushing Blows — +8% crit chance, +50% crit damage
- Sunder Armor — +20% increased damage
- Fleet Footed — increased movement speed
- Defence — +4 maximum HP
- Deadly Precision — +1% instant kill chance
- Cold Blood — if no enemy is within 5 blocks, +5 damage
- Anatomical Knowledge — if main hand weapon is ranged, +10% crit rate
- Crippling Shot — if main hand weapon is ranged, +50% crit damage
- Flood of Anger — if an enemy is within 5 blocks, +5 damage
- Razor Focus — +10% dodge chance
- Domination — increases damage dealt to mobs
- Undying — when killed, restore HP to 50% once every 15 minutes

Figure 4.3 — The Signs tab. Each of the five Signs has its own upgrade column.

Signs Tab Perks

Each of the five Signs has a dedicated upgrade column:

- Aard Intensity — increases Aard knockback force
- Aard Sweep — alternate Sign form, strikes in a radius around the player
- Shock Wave — Aard deals X direct damage
- Igni Intensity — increases Igni burning duration
- Firestream — increases Igni area of effect diameter
- Pyromaniac — Igni deals additional burning damage on tick
- Yrden Intensity — increases Yrden slow potency
- Magic Trap — alternate Sign form
- Sustained Glyphs — increases Yrden duration by 5 seconds
- Quen Intensity — increases Quen maximum absorption
- Exploding Shield — when Quen ends, deal damage to all nearby enemies
- Quen Discharge — deals absorbed damage to the attacker
- Axii Intensity — increases Axii Sign duration
- Delusion — grants Hero of the Village effect

- Domination — increases damage dealt to affected mobs

Figure 4.4 — The Alchemy tab. Brewing, Oil Preparation, Bomb Creation, Mutation, and Trial of Grasses columns.

Alchemy Tab Perks

The Alchemy tab is split into five columns — Brewing, Oil Preparation, Bomb Creation, Mutation, and Trial of Grasses:

- Refreshment — each potion heals 10% of max HP when consumed
- Delayed Recovery — if Toxicity is above 80%, reactivate the Elixir effect
- Side Effects — 33% chance to activate a random secondary potion effect on consumption
- Poisoned Blades — oils give a 5% chance to poison on hit
- Protective Coating — reduces damage taken from the mob type the equipped oil targets
- Hunter Instinct — deals additional damage to the mob type the equipped oil targets
- Pyrotechnics — bombs deal additional damage
- Efficiency — bombs have a chance to remain in inventory after use
- Cluster Bombs — bombs have an increased area of effect
- Acquired Tolerance 1, 2, 3 — each tier increases Maximum Toxicity
- Endure Pain — increases max HP by X when Toxicity is above threshold
- Fast Metabolism — Toxicity drops X points faster per second
- Killing Spree — when Toxicity is above 0, increases crit chance for X seconds (stackable)

General Tab

The General tab contains passive survivability and utility upgrades including maximum HP increases, stamina regen bonuses, and School Set synergy bonuses that activate based on the equipped armor set. These synergies stack on top of each school's base effect, further rewarding full commitment to a playstyle.

4.4 Developer & Debug Commands

Witchercraft registers several custom commands, accessible via the in-game console. These commands are primarily used during development and debugging to inspect and manipulate player stats without relying on normal gameplay progression.

/witchercraft stats	Prints all current player variables (Toxicity, crit chance, crit damage, life steal, etc.) to chat for inspection
/witchercraft levelfix	Corrects any desync in player level or XP values that can occur during testing
/witchercraft devlog	Toggles the Dev Log status effect on the player. With Dev Log active, every damage calculation prints its full breakdown to chat — base damage, combined damage, crit roll, crit result, and life steal amount — enabling live formula verification
/witchercraft resetskills	Resets all Skill Tree investments, returning spent points to the player for redistribution
/witchercraft settoxicity [value]	Directly sets the player's current Toxicity to the specified value for testing threshold behaviours

The Dev Log command is particularly valuable during development — it exposes the full damage calculation pipeline in real time, making it possible to verify that stat changes from skill tree upgrades, gear, and potions are applying correctly without requiring external debugging tools.

4.5 Sign Particle Systems

Each Sign has custom particle effects tied to its cast and hold states. Quen required two distinct particle types — one for the initial cast and one for the sustained hold state — implemented as separate particle classes with independent lifecycle management. Aard and Igni are implemented as entity-based particles (AardParticlesEntity, IgniParticlesEntity), allowing them to move through the world, apply hit detection, and interact with the environment rather than being purely cosmetic.

4.6 Projectile Entity System for Bombs

Each bomb type is implemented as a dedicated entity class with its own physics, trajectory, and hit detection logic. This allows each bomb to behave differently on impact — some create lingering area effects, some apply status effects in a radius, and some interact with other game systems such as fire or Signs. Custom death messages are registered for bomb kills.

4.7 Oils as Weapon Enchantments

One of the most deliberate design decisions in the mod was mapping Witcher Oils to Minecraft's enchantment system rather than implementing them as a separate held-item mechanic. This lets oils feel native to Minecraft — players apply them to weapons in familiar ways — while still capturing the Witcher design intent: preparation and enemy knowledge matter. Each oil enchantment targets a specific monster archetype and scales with Alchemy skill upgrades.

4.8 Custom Armor Renderers

All six Witcher School armor sets use custom 3D armor renderer classes rather than Minecraft's default flat-texture system. Each set has its own renderer class, allowing for unique geometry and texture layering that reflects the distinctive visual identity of each Witcher school.

4.9 Package Architecture

The mod is organised into clean, domain-separated packages:

- client/gui — All custom GUI screens (Sign menu, Pause menu, Alchemy, Character, Bestiary, Meditation, Bestiary entries)
- client/particle — Custom particle implementations
- client/renderer — Custom armor renderers
- client/screens — HUD overlays (Stats, Toxicity, Medallion)
- entity — Projectile and particle entities
- item — Individual item logic classes
- init — Registry classes for Items, Entities, Effects, Menus, Particles, Keybindings, Tabs
- network — Client-server synchronization packets
- potion — Custom mob effect implementations
- procedures — Shared game logic and event procedures (including DamageCalculatorProcedure)
- command — Developer and utility commands
- world/inventory — Custom container and inventory logic

5. Planned Features

The following features are actively planned and in various stages of design or early implementation. They represent the intended full scope of the Witchercraft mod.

5.1 Monster Entity Implementation

The four monsters with completed 3D models (Noonwraith, Fiend, Leshen, Cockatrice) are the immediate next priority. Each will receive full AI behavior, attack patterns, loot tables, spawning rules, and Bestiary integration. The six monsters currently in the Bestiary (Drowner, Foglet, Graveir, Bruxa, Higher Vampire, Rotfiend) will also be implemented as spawnable entities with behaviors faithful to The Witcher 3.

5.2 Contract System

A Witcher Contract system modelled on the notice board quests from The Witcher 3, giving players structured monster-hunting objectives with rewards. This would tie the Bestiary, monster entities, and the economy together into a cohesive gameplay loop.

5.3 Expanded Crafting and Economy

Alchemy ingredient sourcing through monster drops and world exploration, deeper crafting recipes for all alchemical items, and a trading system for Witcher supplies.

5.4 World Generation

Custom structures and points of interest — abandoned villages, monster nests, notice boards, and herbalist huts — that make the Minecraft world feel like a Witcher-appropriate environment to explore.

6. Installation

WitcherCraft requires the following to run:

- Minecraft Java Edition (compatible version)
- NeoForge mod loader installed

To install:

- Download and install NeoForge from the official NeoForge installer.
- Place the WitcherCraft .jar file into your Minecraft /mods/ folder.
- Launch Minecraft using the NeoForge profile.
- Once in-game, use the Sign Gui Keybind to open the Sign selector, and the Pause Menu Keybind to access Character, Alchemy, Bestiary, Glossary, and Meditation.

7. Technologies Used

- Java — Primary programming language
- NeoForge — Minecraft mod loader and API framework
- Minecraft Java Edition API — Game hooks, entity system, item registry, effects, enchantments, particles
- Custom JSON asset pipeline — Item models, equipment definitions, language files
- Client-Server networking — Custom packet system for player stat synchronisation

Witchercraft — In Development